HyperKZG Commitment 구현 상세 분석

개요

HyperKZG는 Kate-Zaverucha-Goldberg (KZG) commitment 스킴을 다변수 다항식으로 확장한 고급 암호학적 commitment 방식입니다. BN254 타원 곡선과 페어링 기반 암호학을 사용하여 효율적인 영지식 증명을 제공하며, GPU 가속을 통해 대규모 데이터 처리를 지원합니다.

수학적 배경

KZG Commitment 스킴

KZG commitment는 다항식 $f(x)$ 에 대해 다음과 같이 정의됩니다:

$$Com(f) = g^{f(\tau)} = g^{(a_0 + a_1 \tau + a_2 \tau^2 + \ldots + a_n \tau^n)}$$

여기서:

- g : 타원 곡선의 생성원
- τ : 신뢰할 수 있는 설정에서 생성된 비밀값
- a_i : 다항식의 계수들

HyperKZG 확장

HyperKZG는 이를 다변수 다항식으로 확장:

$$Com(f) = g^{f(\tau_1, \tau_2, \ldots, \tau_n)}$$

다변수 다항식의 평가를 효율적으로 증명할 수 있습니다.

파일 구조 분석

`crates/proof-of-sql/src/proof_primitive/hyperkzg/commitment.rs`

HyperKZGCommitment 구조체

```
#[derive(Clone, Copy, Debug, PartialEq, Eq, CanonicalSerialize, CanonicalDeseria
pub  struct HyperKZGCommitment {
/// The underlying commitment.
pub commitment: G1Projective,
}
```

구조 분석:

- `G1Projective` : BN254 곡선의 G1 그룹 원소 (사영 좌표계)
- **사영 좌표계**: 효율적인 타원 곡선 연산을 위한 좌표 표현
- **직렬화 지원**: Arkworks의 표준 직렬화 형식 지원
- **기본값**: 무한원점(identity element)으로 초기화

직렬화 구현

```
impl Serialize for HyperKZGCommitment {
fn serialize<S: Serializer>(&self, serializer: S) -> Result<S::Ok, S::Error> {
let affine: G1Affine = self.commitment.into();
match affine.xy() {
```

```
None => ([0u8; 32], [0u8; 32]).serialize(serializer),

Some((x, y)) => {

let  mut x_bytes = [0u8; 32];

CanonicalSerialize::serialize_uncompressed(&x, &mut x_bytes[..])

.map_err(serde::ser::Error::custom)?;

x_bytes.reverse();

let  mut y_bytes = [0u8; 32];

CanonicalSerialize::serialize_uncompressed(&y, &mut y_bytes[..])

.map_err(serde::ser::Error::custom)?;

y_bytes.reverse();

(x_bytes, y_bytes).serialize(serializer)

}

}

}

}
```

직렬화 과정:

1. **좌표 변환**: 사영 좌표를 아핀 좌표로 변환
2. **무한원점 처리**: 특별한 경우로 (0,0) 좌표로 표현
3. **바이트 변환**: x, y 좌표를 각각 32바이트로 직렬화
4. **엔디안 변환**: 바이트 순서를 뒤집어 일관된 형식 보장
5. **튜플 직렬화**: (x_bytes, y_bytes) 형태로 최종 직렬화

역직렬화 구현

```rust
impl<'de> Deserialize<'de> for HyperKZGCommitment {
fn deserialize<D: Deserializer<'de>>(deserializer: D) -> Result<Self, D::Error>
let (mut x_bytes, mut y_bytes) = <([u8; 32], [u8; 32])>::deserialize(deserialize
let affine: G1Affine = if (x_bytes, y_bytes) == ([0u8; 32], [0u8; 32]) {
G1Affine::identity()
} else {
x_bytes.reverse();
y_bytes.reverse();
let x = CanonicalDeserialize::deserialize_uncompressed(&x_bytes[..])
.map_err(serde::de::Error::custom)?;
let y = CanonicalDeserialize::deserialize_uncompressed(&y_bytes[..])
.map_err(serde::de::Error::custom)?;
G1Affine::new_unchecked(x, y)
};
Ok(Self {
commitment: affine.into(),
})
}
}
```

역직렬화 과정:

1. **바이트 추출**: 64바이트 데이터에서 x, y 좌표 분리

2. **무한원점 검사**: (0,0) 패턴 확인

3. **엔디안 복원**: 바이트 순서 원복

4. **좌표 복원**: 바이트에서 필드 원소로 변환

5. **점 생성**: 검증 없이 아핀 점 생성 (성능 최적화)

대수적 연산 구현

덧셈 연산

```
impl AddAssign for HyperKZGCommitment {
fn add_assign(&mut  self, rhs: Self) {
self.commitment = self.commitment + rhs.commitment;
}
}
```

동형성 활용:

- `Com(f) + Com(g) = Com(f + g)`
- 타원 곡선 점 덧셈을 통한 다항식 덧셈 구현

스칼라 곱셈

```
impl Mul<&HyperKZGCommitment> for BNScalar {
type Output = HyperKZGCommitment;
fn mul(self, rhs: &HyperKZGCommitment) -> Self::Output {
Self::Output {
commitment: rhs.commitment * self.0,
```

```
}
}
}
```

스칼라 곱셈:

- `a · Com(f) = Com(a · f)`
- 타원 곡선 스칼라 곱셈을 통한 다항식 스케일링

음수 연산

```
impl Neg for HyperKZGCommitment {
type Output = Self;
fn neg(self) -> Self::Output {
(-BNScalar::ONE) * self
}
}
```

음수 구현:

- `-Com(f) = Com(-f)`
- -1을 곱하여 점의 y 좌표 부호 변경

Commitment 계산 구현

CPU 기반 구현

```
#[cfg(any(not(feature = "blitzar"), test))]
```

```rust
#[tracing::instrument(
name = "compute_commitment_generic_impl (cpu)",
level = "debug",
skip_all
)]
fn compute_commitment_generic_impl<T: Into<BNScalar> + Clone + Sync>(
setup: HyperKZGPublicSetup<'_>,
offset: usize,
scalars: &[T],
) -> HyperKZGCommitment {
assert!(offset + scalars.len() <= setup.len());
let product: G1Projective = if_rayon!(scalars.par_iter(), scalars.iter())
.zip(&setup[offset..offset + scalars.len()])
.map(|(t, s)| *s * Into::<BNScalar>::into(t).0)
.sum();
HyperKZGCommitment {
commitment: G1Projective::from(product),
}
}
```

CPU 구현 특징:

1. **제네릭 설계**: 다양한 데이터 타입을 BNScalar로 변환
2. **병렬 처리**: Rayon을 통한 멀티스레드 처리

3. **메모리 안전성**: 배열 경계 검사

4. **효율적 합산**: Iterator의 sum() 메서드 활용

GPU 기반 구현 (Blitzar)

```
#[cfg(feature = "blitzar")]

#[tracing::instrument(name = "compute_commitments (gpu)", level = "debug", skip_

fn compute_commitments(

committable_columns: &[crate::base::commitment::CommittableColumn],

offset: usize,

setup: &Self::PublicSetup<'_>,

) -> Vec<Self> {

if committable_columns.is_empty() {

return Vec::new();

}


// Find the maximum length of the columns to get number of generators to use

let max_column_len = committable_columns

.iter()

.map(CommittableColumn::len)

.max()

.expect("You must have at least one column");


let  mut blitzar_commitments = vec![G1Affine::default(); committable_columns.len
```

```
blitzar::compute::compute_bn254_g1_uncompressed_commitments_with_generators(
&mut blitzar_commitments,
&slice_ops::slice_cast(committable_columns),
&setup[offset..offset + max_column_len],
);

slice_ops::slice_cast(&blitzar_commitments)
}
```

GPU 구현 특징:

1. **배치 처리**: 여러 컬럼을 동시에 처리
2. **메모리 최적화**: 최대 길이 기준으로 생성원 할당
3. **타입 변환**: slice_cast를 통한 효율적인 타입 변환
4. **GPU 가속**: CUDA 커널을 통한 병렬 연산

컬럼별 처리

```
fn compute_commitments_impl(
committable_columns: &[crate::base::commitment::CommittableColumn],
offset: usize,
setup: &<HyperKZGCommitment as Commitment>::PublicSetup<'_>,
) -> Vec<HyperKZGCommitment> {
if_rayon!(committable_columns.par_iter(), committable_columns.iter())
```

```
.map(|column| match column {
CommittableColumn::Boolean(vals) => {
compute_commitment_generic_impl(setup, offset, vals)
}
CommittableColumn::Uint8(vals) => compute_commitment_generic_impl(setup, offset,
// ... 각 데이터 타입별 처리
CommittableColumn::Decimal75(_, _, vals)
| CommittableColumn::Scalar(vals)
| CommittableColumn::VarChar(vals)
| CommittableColumn::VarBinary(vals) => {
compute_commitment_generic_impl(setup, offset, vals)
}
})
.collect()
}
```

타입별 처리:

- **기본 타입**: 직접 스칼라로 변환
- **복합 타입**: 4-limb 표현으로 변환
- **문자열/바이너리**: 해시를 통한 스칼라 변환
- **병렬 처리**: 각 컬럼을 독립적으로 처리

공개 설정 (Public Setup)

타입 정의

```
/// `PublicSetup` type associated with the `HyperKZG` commitment scheme.
pub type HyperKZGPublicSetup<'a> = &'a [G1Affine];

/// When borrowed, `PublicSetup` type associated with the `HyperKZG` commitment
pub type HyperKZGPublicSetupOwned = Vec<G1Affine>;
```

설정 구조:

- **생성원 배열**: `[g, g^τ, g^τ², ..., g^τⁿ]` 형태의 구조화된 참조 문자열
- **차용 vs 소유**: 메모리 효율성을 위한 차용 타입과 소유 타입 분리
- **아핀 좌표**: 저장 효율성을 위한 압축된 좌표 사용

역직렬화 구현

```
pub fn deserialize_flat_compressed_hyperkzg_public_setup_from_slice(
bytes: &[u8],
validate: Validate,
) -> Result<HyperKZGPublicSetupOwned, SerializationError> {
bytes
.chunks(COMPRESSED_SIZE)
.map(|chunk| G1Affine::deserialize_with_mode(chunk, Compress::Yes, validate))
.collect()
```

```
}
```

역직렬화 과정:

1. **청크 분할**: 32바이트씩 분할하여 각 점 처리
2. **압축 해제**: 압축된 점 형식에서 아핀 좌표 복원
3. **검증 옵션**: 점이 곡선 위에 있는지 선택적 검증
4. **배치 처리**: Iterator를 통한 효율적인 일괄 처리

테스트용 설정 로드

```
#[cfg(feature = "hyperkzg_proof")]
#[must_use]
pub  fn load_small_setup_for_testing() -> (
HyperKZGPublicSetupOwned,
nova_snark::provider::hyperkzg::VerifierKey<super::HyperKZGEngine>,
) {
// 하드코딩된 검증 키 상수들
const VK_X_REAL: [u64; 4] = [0x2a74_74c0_708b_ef80, ...];
// ...
let tau_h = G2Affine {
x: Fq2::new(Fq::from_raw(VK_X_REAL), Fq::from_raw(VK_X_IMAG)),
y: Fq2::new(Fq::from_raw(VK_Y_REAL), Fq::from_raw(VK_Y_IMAG)),
};
}
```

테스트 설정:

- **고정 매개변수**: 재현 가능한 테스트를 위한 하드코딩된 값들
- **G2 원소**: 페어링 검증을 위한 G2 그룹의 검증 키
- **파일 로드**: 미리 계산된 신뢰할 수 있는 설정 파일 사용

BN254 스칼라 구현

타입 정의

```
/// The scalar used in the `HyperKZG` PCS. This is the BN254 scalar.
pub  type BNScalar = MontScalar<ark_bn254::FrConfig>;
```

스칼라 특성:

- **Montgomery 형태**: 효율적인 모듈러 연산을 위한 Montgomery 표현
- **BN254 필드**: 254비트 소수 필드 `r = 21888242871839275222246405745257275088548364400416034343698204186575808495617`
- **Fr 구성**: BN254 곡선의 스칼라 필드

Nova 호환성

```
#[test]
#[cfg(feature = "hyperkzg_proof")]
fn we_can_convert_from_posql_scalar_to_nova_scalar() {
// Test zero
assert_eq!(NovaScalar::from(0_u64), NovaScalar::from(BNScalar::ZERO));
```

```
// Test arithmetic operations

for _ in  0..10 {

let a = BNScalar::rand(&mut rng);

let b = BNScalar::rand(&mut rng);

assert_eq!(

NovaScalar::from(a + b),

NovaScalar::from(a) + NovaScalar::from(b)

);

assert_eq!(

NovaScalar::from(a * b),

NovaScalar::from(a) * NovaScalar::from(b)

);

}

}
```

호환성 검증:

- **기본 상수**: 0, 1, -1 등의 기본값 일치 확인
- **연산 보존**: 덧셈과 곱셈 연산이 변환 후에도 일치
- **무작위 테스트**: 다양한 값에 대한 일관성 검증

Commitment 평가 증명

구조체 정의

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct HyperKZGCommitmentEvaluationProof {
com: Vec<HyperKZGCommitment>,
v: Vec<[BNScalar; 3]>,
w: [HyperKZGCommitment; 3],
}
```

증명 구성 요소:

- `com` : 중간 commitment들의 벡터
- `v` : 각 라운드의 평가값들 (3개씩)
- `w` : 최종 witness commitment들

Nova 통합

```rust
impl From<&HyperKZGCommitmentEvaluationProof> for EvaluationArgument<HyperKZGEng
fn from(value: &HyperKZGCommitmentEvaluationProof) -> Self {
let nova_com = value.com.iter().map(Into::into).collect();
let nova_w = value.w.map(Into::into);
let nova_v = value.v.iter().map(|vj| vj.map(Into::into)).collect();
EvaluationArgument::new(nova_com, nova_w, nova_v)
}
}
```

Nova 호환성:

- **타입 변환**: Proof of SQL 타입을 Nova 타입으로 변환
- **구조 보존**: 증명 구조를 그대로 유지
- **양방향 변환**: Nova에서 다시 Proof of SQL로 변환 가능

성능 최적화

GPU 가속

1. **Blitzar 통합**: CUDA 기반 병렬 연산
2. **배치 처리**: 여러 commitment를 동시에 계산
3. **메모리 최적화**: GPU 메모리 사용량 최소화

CPU 최적화

1. **병렬 처리**: Rayon을 통한 멀티스레드 활용
2. **SIMD 연산**: 벡터화된 필드 연산
3. **메모리 지역성**: 캐시 친화적인 데이터 접근

직렬화 최적화

1. **압축 형식**: 32바이트 압축된 점 표현
2. **배치 직렬화**: 여러 점을 효율적으로 처리
3. **검증 옵션**: 필요에 따른 선택적 검증

보안 특성

암호학적 가정

1. **이산 로그 문제**: BN254 곡선에서의 이산 로그 문제의 어려움

2. **페어링 가정**: 페어링 함수의 일방향성

3. **신뢰할 수 있는 설정**: 초기 설정의 무결성

구현 보안

1. **상수 시간 연산**: 타이밍 공격 방지

2. **메모리 보안**: 민감한 데이터의 안전한 처리

3. **검증 로직**: 모든 입력에 대한 적절한 검증

이 HyperKZG commitment 구현은 Proof of SQL의 고성능 암호학적 기반을 제공하며, GPU 가속과 최적화된 구현을 통해 실용적인 영지식 증명을 가능하게 합니다.